“SmartRecipes”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Kolavali Sanjana	**:2520030614**
Yerrolla Eekshitha	**:2520030615**

Team Number: 22

Section:6

Under the Guidance of

Dr. V.Sireesha
Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **SmartRecipes** is a record of Bonafide work of **Kolavali Sanjana – 2520030614, Yerrolla Eekshitha – 2520030615,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Kolavali Sanjana – 2520030614

Yerrolla Eekshitha – 2520030615

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"SmartRecipes"** is a bonafide work done and submitted by **Kolavali Sanjana – 2520030614, Yerrolla Eekshitha – 2520030615,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

Description in Times New Roman font 12 size with two to three paragraphs thanking guide Dr. S. Vinay Kumar, Associate Professor, Dept. of CSE and Dr. P. Venkateshwara Rao, Associate Professor, HOD CSE

Kolavali Sanjana – 2520030614

Yerrolla Eekshitha – 2520030615

Abstract

The **SmartRecipes: Intelligent Recipe Recommendation System** is a web-based application designed to help users discover recipes based on the ingredients available to them. The main objective of the project is to simplify meal planning, reduce food waste, and make cooking more convenient through an interactive and user-friendly interface.

The website provides several features, including ingredient-based recipe search, recipe filtering, detailed cooking instructions, ingredient substitution suggestions, favourites, recipe ratings, random recipe selection, and the ability to add custom recipes. Users can explore different food categories and identify suitable dishes according to their preferences and available ingredients. The application uses HTML, CSS, and JavaScript to provide an interactive experience with a visually appealing beige-and-brown theme. Browser local storage is used to retain selected user data, such as favourite recipes, ratings, and custom recipes.

The project is developed alongside a Java-based implementation that demonstrates concepts from the six modules of the Data Structures and Algorithms (DSA-3) course. These include string-processing techniques, dynamic programming, graph algorithms, approximation methods, randomized algorithms, and related computational techniques. The Java implementation provides an academic foundation for exploring how algorithmic methods can support search, similarity analysis, optimization, and decision-making problems. The website and Java application currently operate as separate components.

Overall, SmartRecipes combines practical web development with data-structure and algorithm concepts to provide an accessible recipe discovery platform. Future enhancements may include database integration, personalized recommendations, nutritional analysis, user accounts, and connecting the website to the Java recommendation engine.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	16
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements	17
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer	18-19
4	Implementation	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 …….	20-21
5	Results		22-23
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	24
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	25-26
	References	- Books, tutorials, APIs, documentation sites used	27-28

List of Figures

1.1	Block Diagram of a Recommendation System	9
1.2	Components of a Recommendation System	10
1.3	Flow Chart to show the working of a Recommendation System	11
1.4	Phases of a Recommendation System	12
1.5	User-User KNN based Recommendation System	13
1.6	Item-Item KNN based Recommendation System	14
1.7	Matrix Factorization based Recommendation System	15

1.1 Block Diagram of a Recommendation System

The block diagram illustrates the overall working of a recommendation system, showing how user inputs are processed to generate relevant recipe recommendations. The process begins with user information such as search queries, available ingredients, preferences, and ratings. The input processing stage cleans and normalizes the data and extracts useful features. The recommendation engine then analyzes the processed information using suitable recommendation approaches and ranks the results. The recipe database provides information about recipes, ingredients, and categories, while user data supplies preferences, search history, ratings, and favourites. Finally, the system presents a ranked list of recommended recipes, including recipe details, similar recipes, and ingredient substitution suggestions.

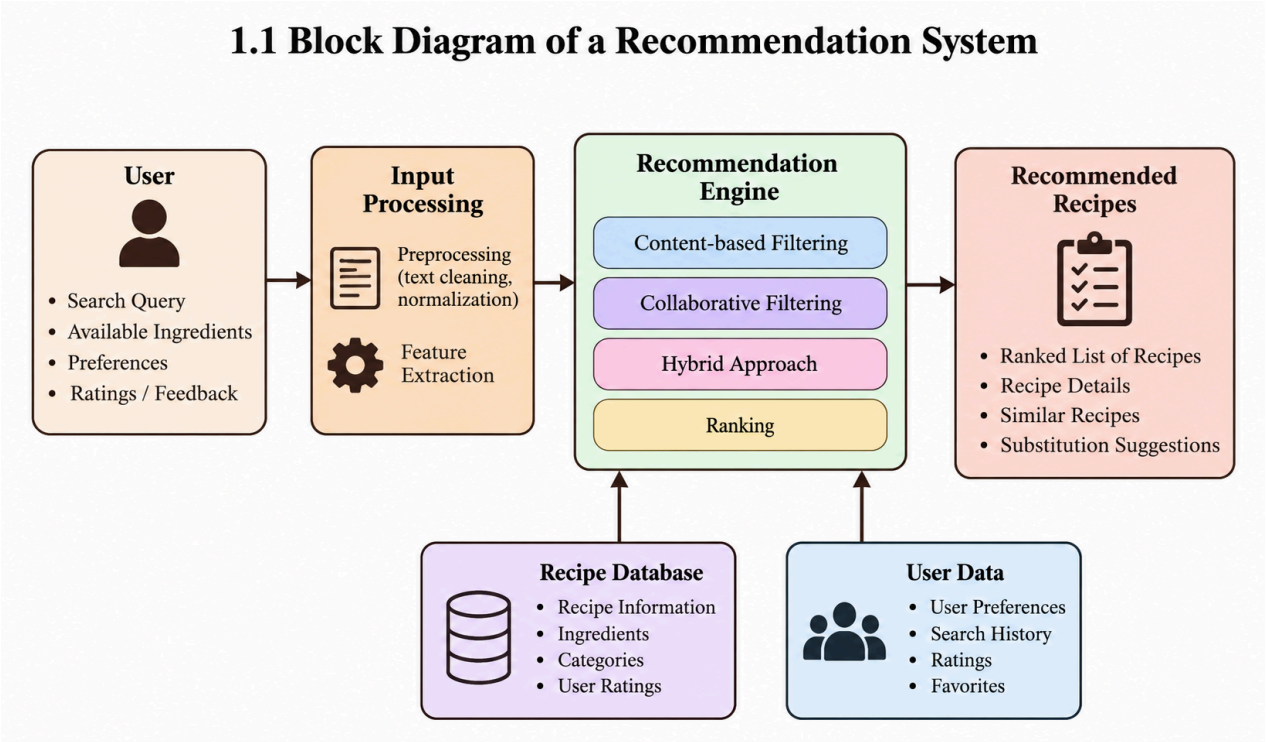


Fig. 1.1 Architecture of Cloud

1.2 Components of a Recommendation System

A recommendation system consists of several components that work together to identify user requirements, process information, and suggest relevant items. In the SmartRecipes project, these components help users discover recipes based on available ingredients and preferences.

Main Components

1. User Interface: Provides the interface through which users search for recipes, enter ingredients, apply filters, and explore recommendations.

2. User Input and Preferences: Collects information such as search queries, available ingredients, recipe categories, and user selections.

3. Recipe Database: Stores the recipe information used by the website, including recipe names, ingredients, categories, and preparation instructions.

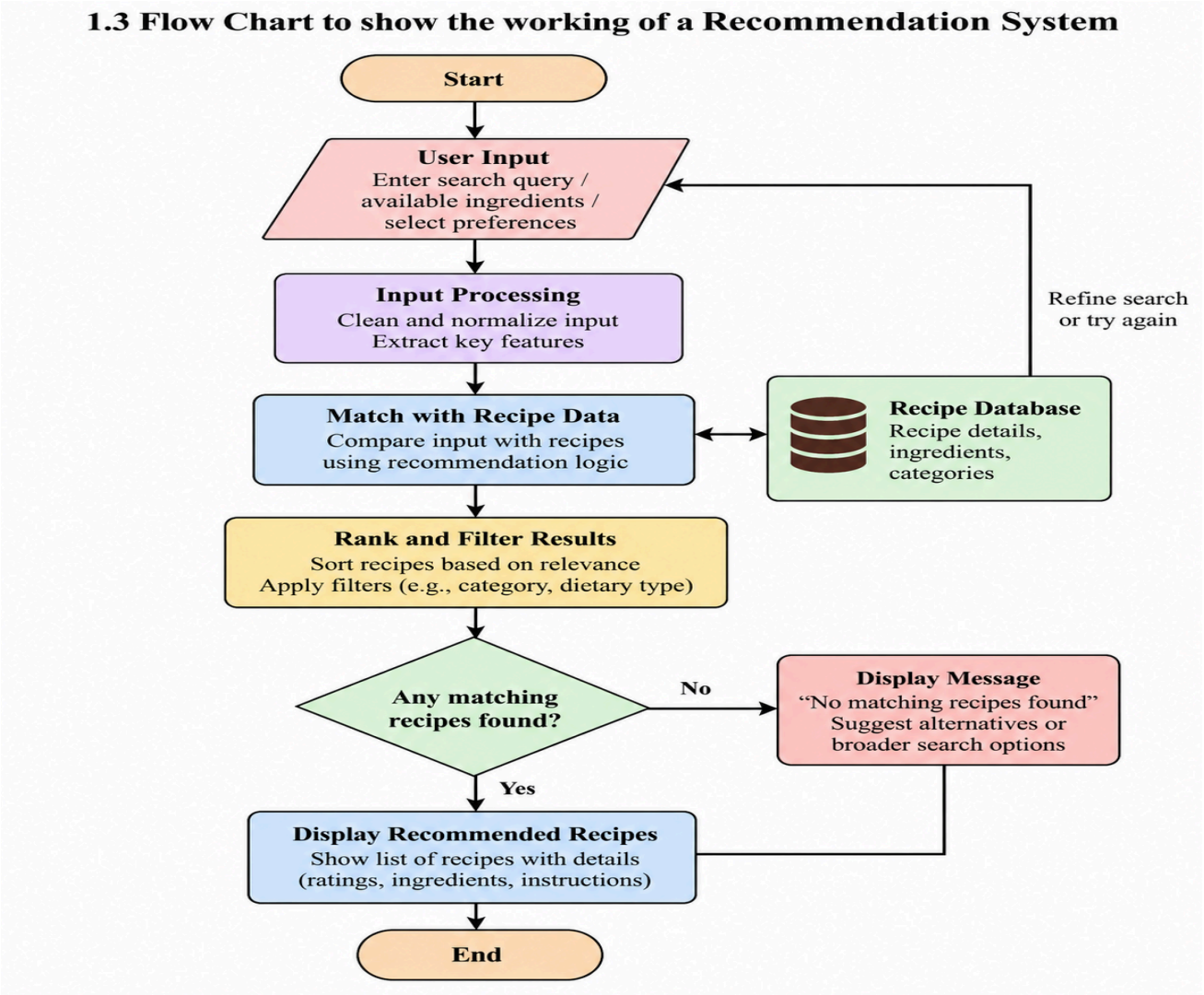
4. Data Processing: Processes user input and prepares it for searching and matching against the available recipes.

5. Recommendation Engine: Evaluates the available recipes according to the implemented search and ingredient-matching logic and identifies relevant results.

6. Ranking and Filtering: Helps users narrow down recipe results using supported categories and search criteria.

7. Results Display: Presents the matching recipes and their details to the user.

1.3 Flow Chart to show the working of a Recommendation System

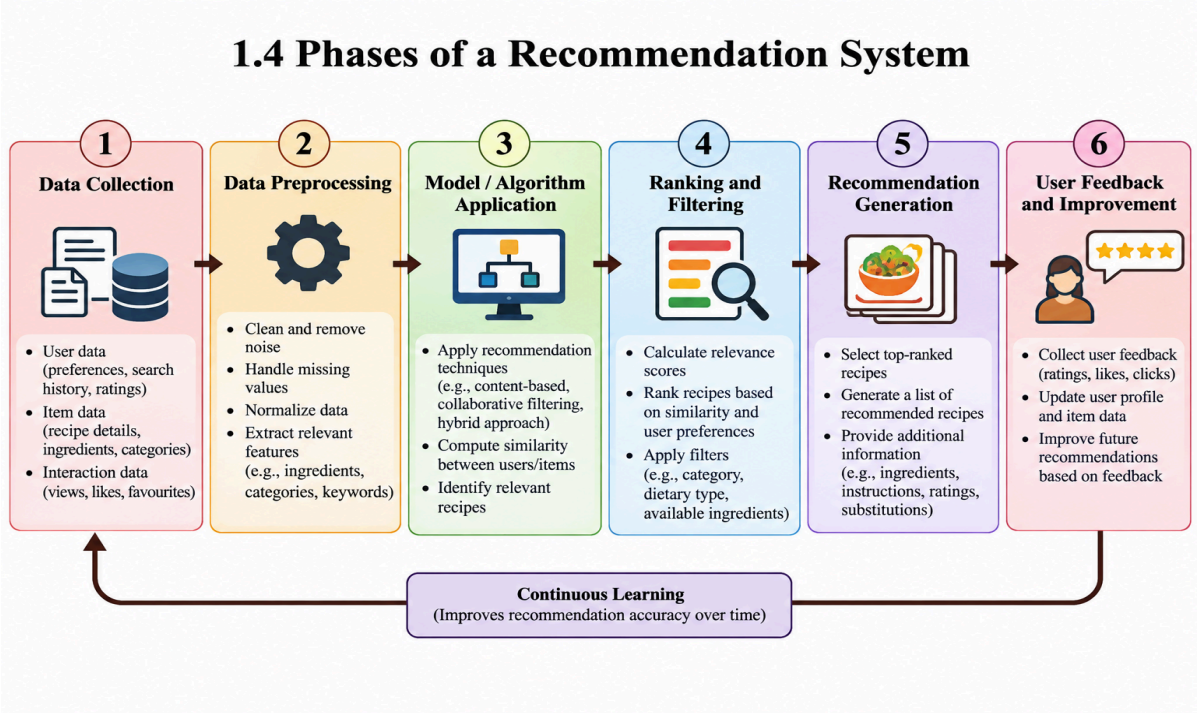


1.4 Phases of a Recommendation System

The diagram illustrates the six major phases of a recommendation system: data collection, data preprocessing, model or algorithm application, ranking and filtering, recommendation generation, and user feedback. Initially, the system collects user preferences and recipe information.

The collected data is then processed to remove inconsistencies and identify relevant features. Next, the recommendation technique analyzes the available information to identify suitable recipes. The results are ranked and filtered according to their relevance before being presented to the user.

Finally, user feedback, such as ratings and preferences, can help improve future recommendations. These phases describe the general recommendation-system workflow; the current SmartRecipes website implements a simpler browser-based search and matching process.



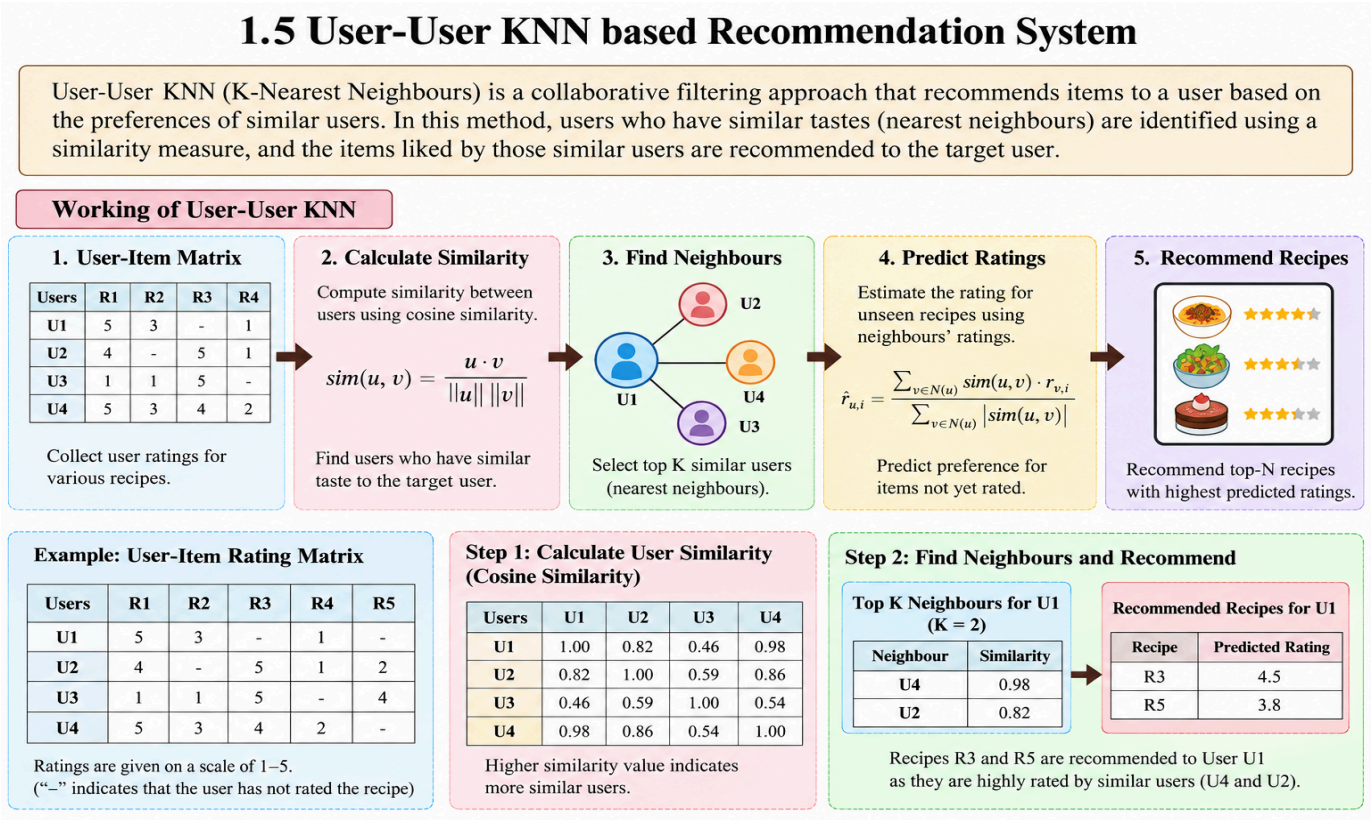
1.5 User–User KNN-Based Recommendation System

The User–User K-Nearest Neighbors (KNN) recommendation system is a collaborative filtering technique that recommends items by identifying users with similar preferences. It assumes that users who have liked or rated similar items in the past may also be interested in similar items in the future.

In a recipe recommendation system, this technique can identify users who have rated similar recipes and use their preferences to suggest recipes that the current user has not tried.

Working Principle

1. Collect user ratings: Gather ratings or preference information for different recipes.
2. Calculate user similarity: Compare users based on their ratings or interactions with recipes.
3. Identify nearest neighbours: Select the K users whose preferences are most similar to the target user.
4. Generate recommendations: Identify recipes liked by similar users that the target user has not tried.
5. Rank the results: Order candidate recipes according to their predicted relevance or preference score.



1.6 Item–Item KNN-Based Recommendation System

Description

Item–Item K-Nearest Neighbors (KNN) is a collaborative filtering technique that recommends items based on similarities between them. In the SmartRecipes context, recipes are compared using information such as user ratings and interactions. When a user likes a particular recipe, the system identifies other recipes that are similar and recommends them.

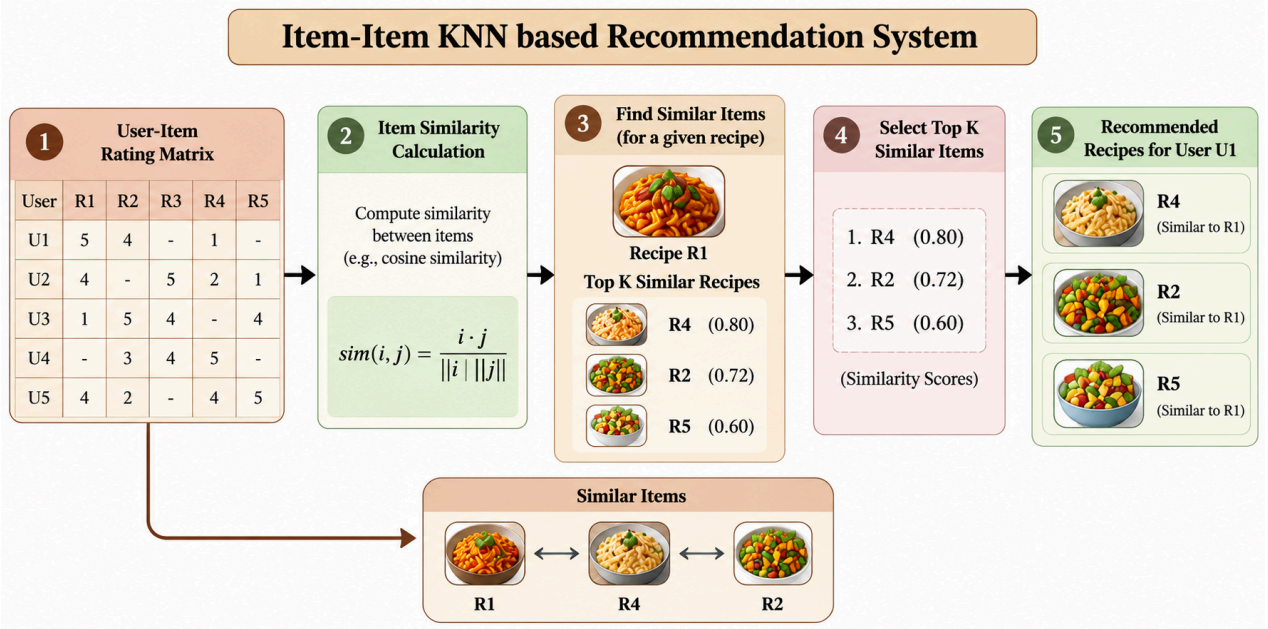
The method first collects user–recipe rating data and calculates the similarity between recipes. It then identifies the top K most similar recipes and recommends suitable items that the user has not tried.

Working steps:

1. Collect user ratings and recipe interaction data.
2. Calculate similarity between recipes using a suitable similarity measure.
3. Identify the nearest matching recipes.
4. Recommend similar recipes to the user.
5. Rank the results according to their similarity scores.

Advantages: Provides recommendations based on similar recipes and can help users discover related dishes.

Limitations: Requires sufficient rating data, and similarity calculations can become expensive when the recipe collection is very large.

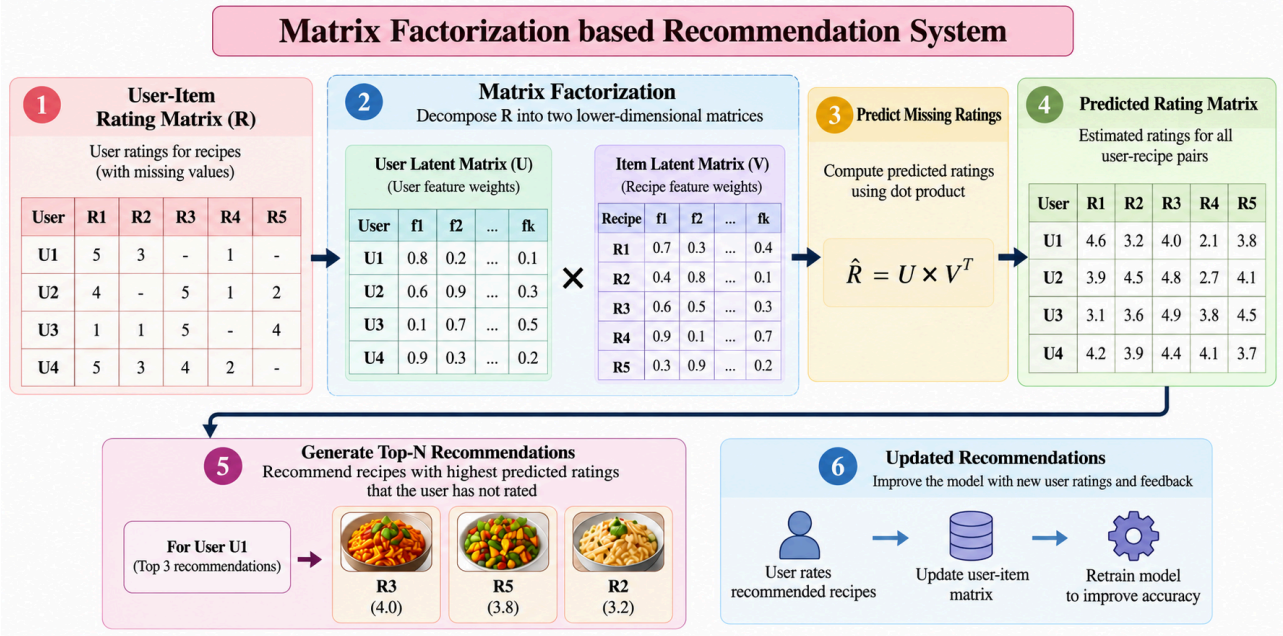


1.7 Matrix Factorization-Based Recommendation System

Description:

Matrix Factorization is a collaborative filtering technique that decomposes a user–item rating matrix into two smaller matrices representing user preferences and recipe characteristics. These matrices capture hidden patterns, such as food preferences, cuisine types, and cooking styles.

The system combines the user and recipe factors to predict ratings for recipes that a user has not rated. Finally, it ranks the predicted ratings and recommends the top recipes to the user. This approach can generate personalized recommendations when sufficient user–recipe interaction data is available.



CHAPTER 1: INTRODUCTION

1.1 Overview

SmartRecipes is a web-based recipe recommendation system designed to make recipe discovery and meal planning easier. It allows users to search for recipes, find dishes based on available ingredients, apply category filters, view cooking instructions, save favourites, rate recipes, and add custom recipes. The website uses HTML, CSS, and JavaScript to provide an interactive interface with a beige-and-brown theme. Browser local storage is used to retain selected user information.

1.2 Problem Statement

- Finding suitable recipes from the ingredients available at home can be difficult and time-consuming.
- Users may struggle to identify recipes that match their food preferences and dietary requirements.
- Searching through numerous recipes manually makes meal planning inconvenient.
- A lack of ingredient substitution suggestions can make cooking difficult when certain ingredients are unavailable.
- Food ingredients may be wasted when users cannot identify suitable dishes to prepare with them.
- Users need a simple platform to search, filter, compare, and explore different recipes.
- Managing favourite recipes and maintaining personal recipe collections can be inconvenient without a dedicated system.
- Therefore, there is a need for an interactive recipe recommendation system that simplifies recipe discovery, supports ingredient-based searching, and improves the overall meal-planning experience.

1.3 Scope of the Project

The project focuses on developing an interactive website for recipe discovery and basic meal planning. It includes ingredient-based searching, recipe filters, recipe details, favourites, ratings, ingredient substitutions, random recipe selection, and custom recipe addition. A separate Java application demonstrates selected Data Structures and Algorithms concepts related to the DSA-3 course.

1.4 Objectives

The main objectives of the SmartRecipes project are:

- To develop a user-friendly recipe discovery website.
- To help users find recipes using available ingredients.
- To provide search, filtering, and recipe-detail features.
- To support favourites, ratings, and custom recipes.
- To simplify meal planning and reduce unnecessary food waste.
- To demonstrate selected DSA-3 concepts through a separate Java application.

CHAPTER 2 : SYSTEM REQUIREMENTS

2.1 Functional Requirements

1. Recipe Search: The system allows users to search for recipes using the available search functionality.
2. Ingredient-Based Search: Users can enter available ingredients to find suitable recipes.
3. Recipe Filtering: Users can filter recipes by categories such as vegetarian, non-vegetarian, breakfast, and quick recipes.
4. Recipe Details: The system displays recipe information, including ingredients and preparation instructions.
5. Favourite Management: Users can save and revisit their favourite recipes.
6. Recipe Ratings: Users can rate recipes based on their preferences.
7. Custom Recipe Addition: Users can add their own recipes through a form.
8. Ingredient Substitution: The system suggests alternative ingredients when selected ingredients are unavailable.
9. Random Recipe Selection: The “Surprise Me” feature selects a random recipe from the available collection.
10. Data Persistence: The system uses browser local storage to retain favourites, ratings, and custom recipes.

2.2 Non-Functional Requirements

1. Usability: The website should have a simple, intuitive interface that users can navigate easily.
2. Performance: Search, filtering, and other interactions should respond promptly for the available recipe collection.
3. Reliability: The application should perform its supported operations consistently and handle user inputs appropriately.
4. Responsiveness: The website should adapt to desktop, tablet, and mobile screen sizes.
5. Maintainability: The code should be organized so that features can be modified and improved easily.
6. Compatibility: The website should work in modern web browsers that support HTML, CSS, JavaScript, and local storage.
7. Data Persistence: Saved favourites, ratings, and custom recipes should remain available after page reloads, provided browser storage is retained.
8. Portability: The website should run locally in a compatible browser without requiring a separate backend server.
9. Visual Consistency: The interface should maintain a consistent beige-and-brown theme, with readable text and clearly identifiable controls.
10. Scalability: The application structure should allow future enhancements, such as database integration and a larger recipe collection.

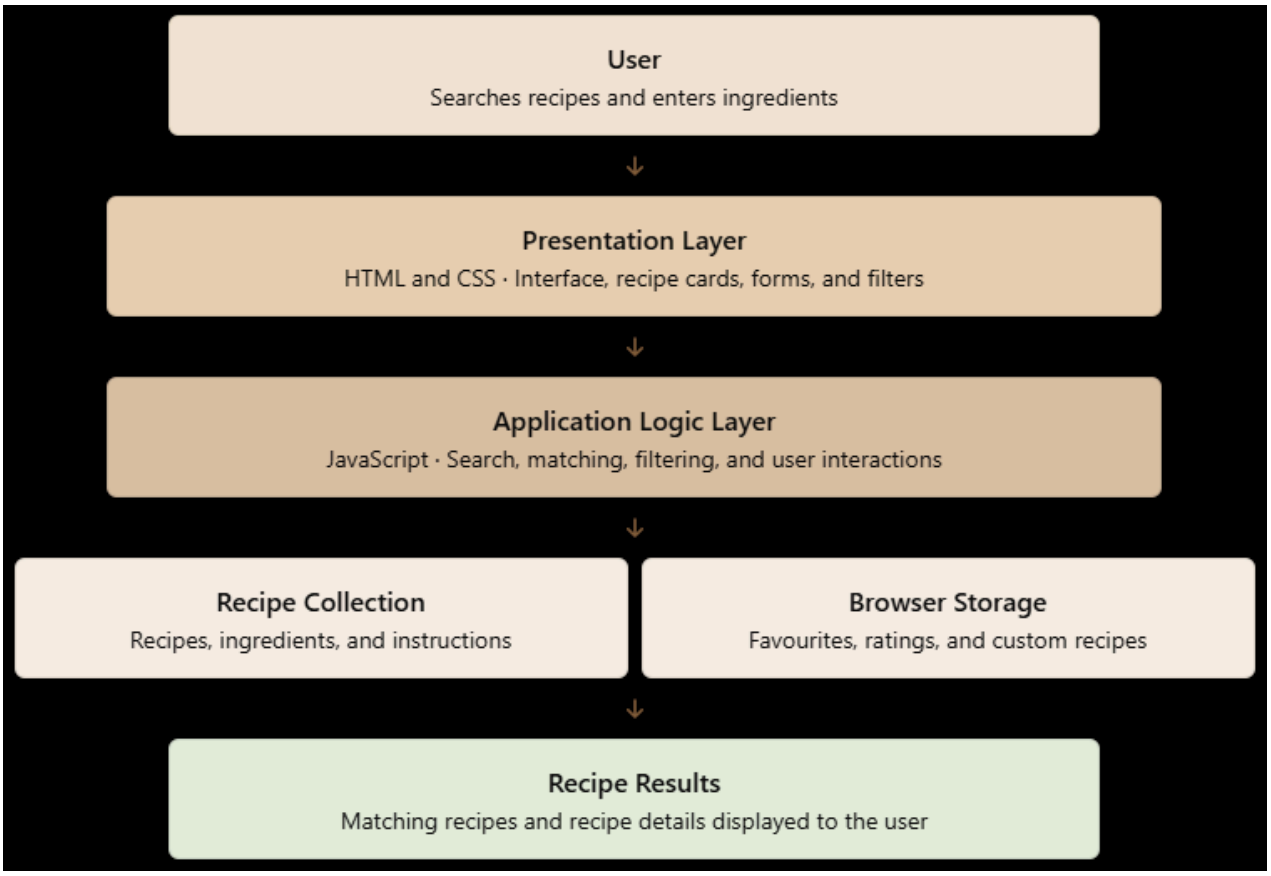
CHAPTER 3 : SYSTEM DESIGN

3.1 High-Level Architecture Diagram

The high-level architecture of the SmartRecipes system illustrates how users interact with the website and how different components work together to provide recipe recommendations. The architecture consists of the presentation layer, application logic layer, recipe data layer, and browser storage layer.

The user interacts with the website through the interface, where they can search for recipes, enter available ingredients, apply filters, and explore recipe details. The application logic processes these actions and matches the input against the available recipe collection. The results are then displayed to the user, while browser local storage retains selected information such as favourites, ratings, and custom recipes.

Figure 3.1: High-Level Architecture of the SmartRecipes System



3.2 Description of Each Layer

1. Presentation Layer

The presentation layer provides the user interface of the SmartRecipes website. It is developed using HTML and CSS and displays the home page, recipe cards, search fields, category filters, and forms. It ensures that users can navigate the website easily.

2. Application Logic Layer

This layer is implemented using JavaScript and manages the interactive features of the website. It processes search queries, matches ingredients with recipes, applies category filters, handles favourites and ratings, and manages custom recipe submissions.

3. Recipe Data Layer

The recipe data layer contains the predefined collection of recipes available on the website. It stores information such as recipe names, ingredients, categories, and preparation instructions, which are used to display relevant results to users.

4. Browser Storage Layer

This layer uses browser local storage to retain user-specific information, including favourite recipes, ratings, and custom recipes. The saved information can be accessed again after reloading the website, provided the browser data has not been cleared.

5. Output Layer

The output layer presents the results generated by the application. It displays matching recipes, filtered results, recipe details, and other relevant information based on the user's input, helping users discover suitable dishes and plan their meals.

CHAPTER 4 : IMPLEMENTATION

4.1 Module-Wise Implementation

The SmartRecipes project is divided into functional modules, with each module responsible for a specific feature of the application.

1. Home Page Module: Provides the main interface with navigation and access to the website's features. The beige-and-brown theme creates a consistent visual appearance.

2. Recipe Search Module: Allows users to search the available recipe collection and find relevant dishes.

3. Ingredient Matching Module: Compares ingredients entered by the user with recipe ingredients to identify suitable recipes.

4. Recipe Filtering Module: Enables users to filter recipes by categories such as vegetarian, non-vegetarian, breakfast, and quick recipes.

5. Recipe Details Module: Displays recipe information, ingredients, and preparation instructions.

6. Favourites and Ratings Module: Allows users to save favourite recipes and rate dishes, retaining this information through browser local storage.

7. Custom Recipe Module: Provides a form for users to add their own recipes to the collection.

8. Ingredient Substitution Module: Suggests alternative ingredients when a selected ingredient is unavailable.

9. Surprise Me Module: Selects a random recipe to help users discover new dishes.

10. Java DSA-3 Module: Contains separate Java source files demonstrating selected algorithms and concepts related to string processing, dynamic programming, graph algorithms, approximation, and randomized algorithms.

4.2 Front-End Logic (UI Rendering and State Management)

The front end is developed using HTML, CSS, and JavaScript. HTML defines the page structure, CSS controls the layout and appearance, and JavaScript handles user interactions.

The interface displays recipe cards, search fields, category filters, recipe details, and custom recipe forms. JavaScript processes user actions, updates the displayed results, and manages features such as favourites, ratings, and recipe addition.

Browser local storage is used to retain selected information, including favourites, ratings, and custom recipes. This allows the information to remain available after reloading the website, provided the stored browser data is not cleared.

The responsive design allows the interface to adapt to different screen sizes, making the website accessible on desktops, tablets, and mobile devices.

4.3 API Integration

The current SmartRecipes website does not use an external recipe API or connect directly to the Java application. Instead, it uses a predefined recipe collection and browser-side JavaScript to provide its main functionality.

The general workflow is:

1. The user enters a search query or available ingredients.
2. JavaScript processes the input.
3. The application checks the available recipe collection.
4. Matching recipes are displayed on the interface.
5. Favourites, ratings, and custom recipes are retained using browser local storage.

In a future version, an API could connect the website to a backend and database, allowing centralized recipe storage, user accounts, and integration with the Java recommendation engine.

4.4 Data Storage and Algorithm Implementation

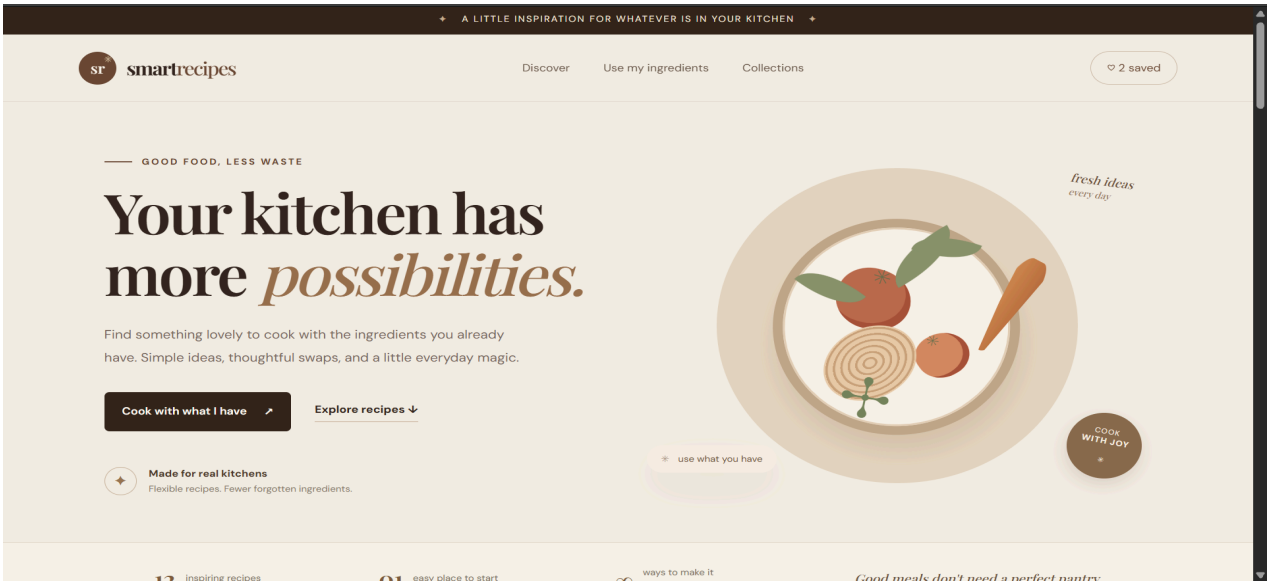
The website stores its predefined recipes in browser-accessible application data and uses local storage to retain favourites, ratings, and custom recipes. This approach allows the current version to run without a separate backend server.

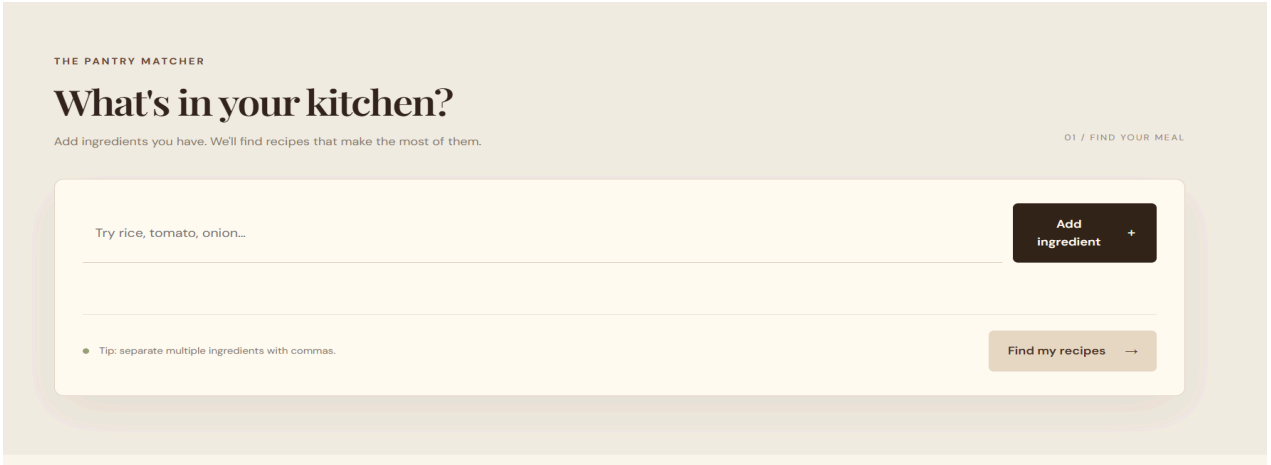
The separate Java application contains selected algorithm implementations for academic demonstration. Representative examples include:

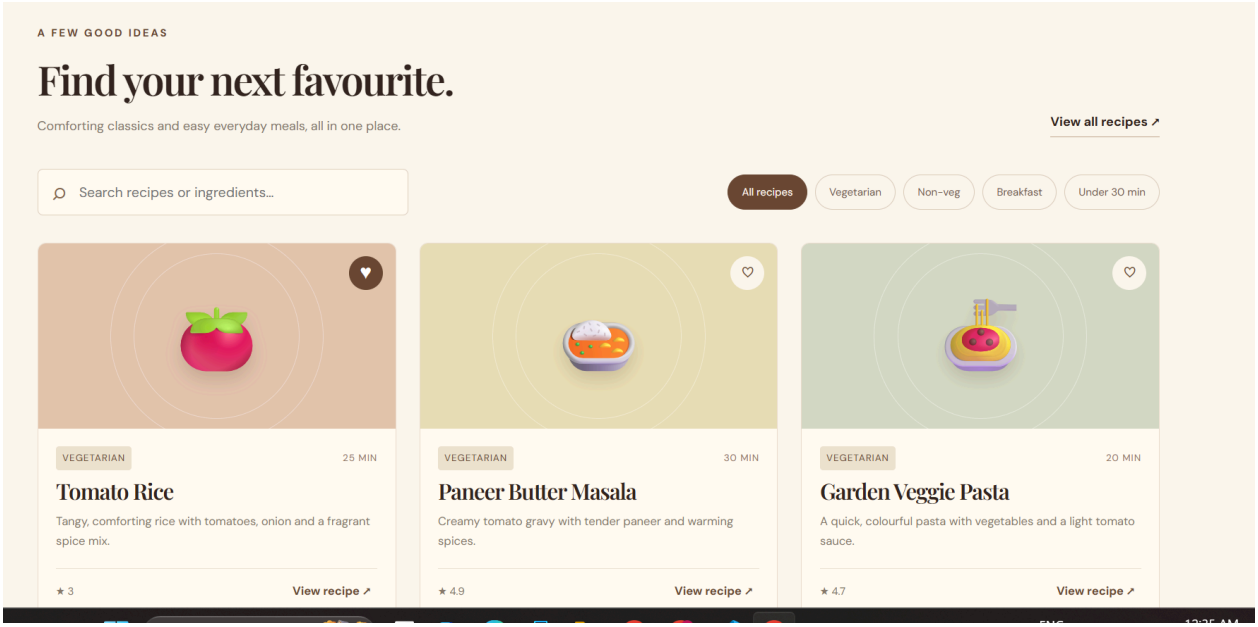
- `KMP.java` — string pattern matching.
- `Levenshtein.java` — edit-distance calculation for comparing strings.
- `RabinKarp.java` — string searching using hashing.
- `MaxFlow.java` — maximum-flow computation.
- `RandomizedQuickSort.java` — randomized sorting.

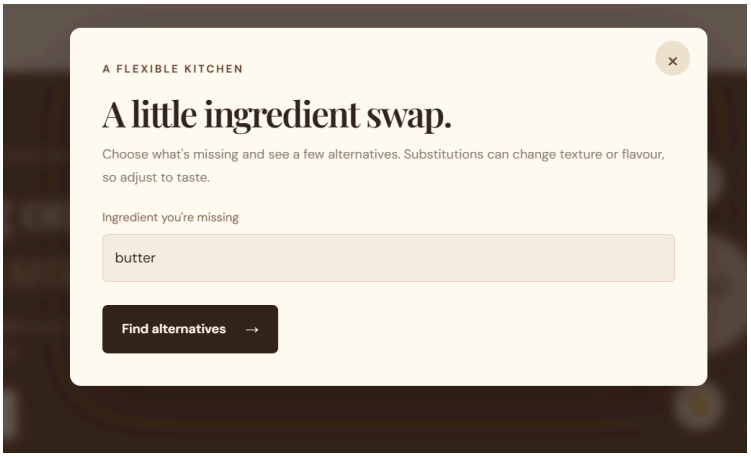
These Java files demonstrate DSA-3 concepts independently. They should not be described as powering the website's features unless they are explicitly connected through the implementation.

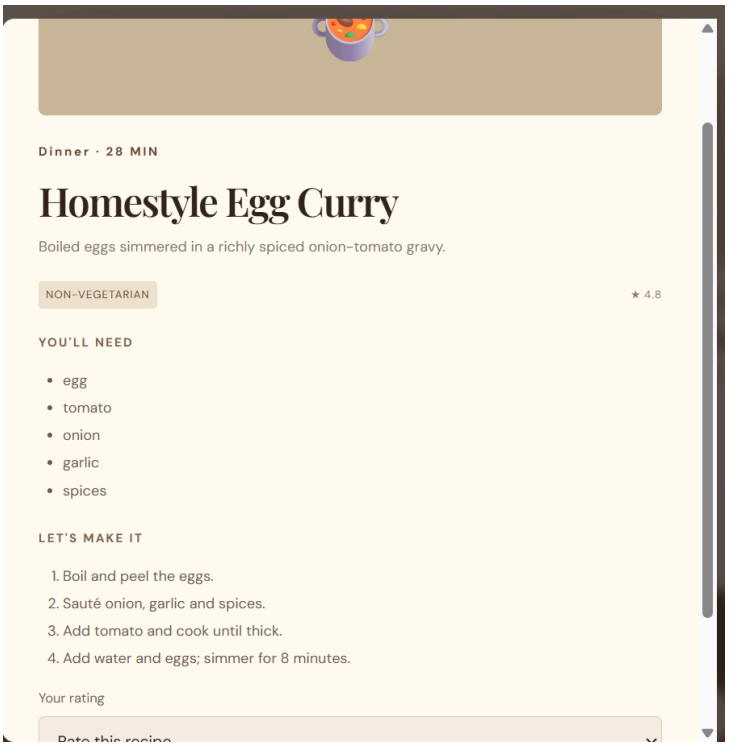
CHAPTER 5 : RESULTS

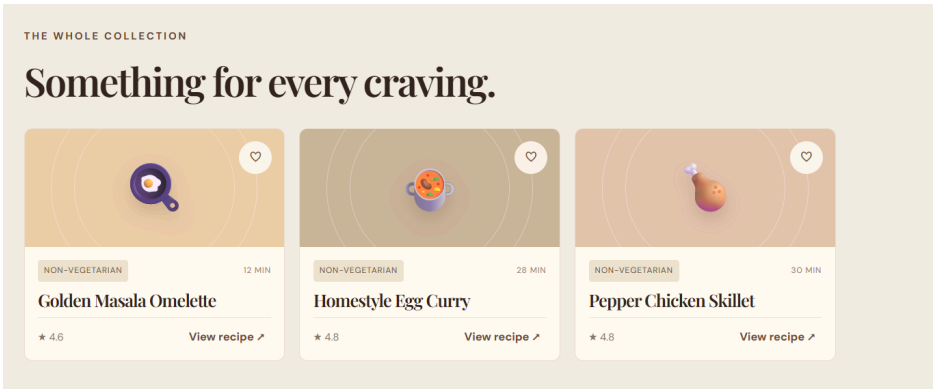







A FLEXIBLE KITCHEN
A little ingredient swap.
Choose what's missing and see a few alternatives. Substitutions can change texture or flavour, so adjust to taste.
Ingredient you're missing
butter
Find alternatives →


Dinner · 28 MIN
Homestyle Egg Curry
Boiled eggs simmered in a richly spiced onion-tomato gravy.
NON-VEGETARIAN
★ 4.8
YOU'LL NEED
egg
tomato
onion
garlic
spices
LET'S MAKE IT
1. Boil and peel the eggs.
2. Sauté onion, garlic and spices.
3. Add tomato and cook until thick.
4. Add water and eggs; simmer for 8 minutes.
Your rating


THE WHOLE COLLECTION
Something for every craving.
NON-VEGETARIAN
12 MIN
Golden Masala Omelette
★ 4.6
View recipe ↗
NON-VEGETARIAN
28 MIN
Homestyle Egg Curry
★ 4.8
View recipe ↗
NON-VEGETARIAN
30 MIN
Pepper Chicken Skillet
★ 4.8
View recipe ↗

CHAPTER 6: CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

During the development of the SmartRecipes project, several challenges were encountered while implementing the website and its interactive features.

- Recipe Search: Processing user queries and matching ingredients with available recipes required careful implementation.
- User Interaction: Managing favourites, ratings, and custom recipe submissions required consistent handling of application data.
- Data Persistence: Ensuring that saved information remained available after reloading the website required browser local storage.
- Responsive Design: Maintaining a consistent layout across different screen sizes required adjustments to the CSS.
- Algorithm Implementation: Understanding and implementing selected DSA-3 algorithms in Java required debugging and testing.

6.2 Solutions Applied

The following approaches were used to address the development challenges.

- Improved Search Logic: JavaScript was used to process search input and identify relevant recipes.
- Organized Code: The application was divided into separate functions and modules to simplify development and maintenance.
- Browser Storage: Local storage was used to retain favourites, ratings, and custom recipes.
- Responsive Layout: CSS styling was adjusted to support different screen sizes.
- Testing and Debugging: Individual features and Java implementations were checked to identify and correct errors.

6.3 Limitations of the System

Although SmartRecipes provides useful recipe discovery features, the current version has certain limitations.

- Limited Recipe Collection: The website uses a predefined collection rather than a large online recipe database.
- No Backend Integration: The website and Java DSA-3 application operate independently.
- Local Data Storage: Saved information is not automatically synchronized across devices.
- No User Authentication: The current website does not provide a complete login or account-management system.
- Basic Recommendations: Recipe suggestions use the implemented search and matching logic rather than a trained machine-learning model.
- Limited Nutritional Information: Comprehensive nutritional analysis and personalized diet planning are not included.

These limitations provide opportunities for future improvements, including database integration, advanced recommendation techniques, user authentication, and connecting the website to the Java application.

CHAPTER 7 : CONCLUSION & FUTURE WORK

7.1 Conclusion

The SmartRecipes: Intelligent Recipe Recommendation System was developed to simplify recipe discovery and meal planning through an interactive and user-friendly website. The application provides features such as ingredient-based recipe search, category filtering, recipe details, favourites, ratings, ingredient substitutions, random recipe selection, and custom recipe addition.

The website is developed using HTML, CSS, and JavaScript, with browser local storage used to retain selected user information. Its beige-and-brown theme provides a consistent and attractive interface. The separate Java application demonstrates selected concepts from the DSA-3 syllabus, providing practical experience in algorithm implementation.

Overall, the project combines web development and algorithmic concepts to create a useful recipe discovery platform. It also provides a foundation for future improvements, including database integration and more advanced recommendation techniques.

7.2 Future Work

The SmartRecipes system can be enhanced in several ways to improve its functionality and usability.

- Database Integration: Connect the application to a database such as MySQL to store recipes and user information.
- Backend Development: Develop a backend service to manage data and application requests.
- Advanced Recommendations: Implement personalized recommendations based on user preferences, ratings, and previous activity.
- Java Integration: Connect the website with the Java application through an API.
- Nutritional Analysis: Add calorie, protein, carbohydrate, and fat information for recipes.
- User Authentication: Introduce secure registration and login functionality.
- Expanded Recipe Collection: Integrate external recipe APIs to provide more dishes and cooking instructions.
- Meal Planning: Add a weekly calendar to organize breakfast, lunch, and dinner.

These enhancements could make SmartRecipes more personalized, scalable, and suitable for real-world use.

7.3 Skills Learned During Development

Developing the SmartRecipes project provided practical experience in web development, programming, problem-solving, and software documentation.

Technical Skills

- HTML for structuring web pages.
- CSS for styling and responsive layouts.
- JavaScript for interactive features and event handling.

- Browser local storage for retaining user data.
- Java programming and debugging.
- Understanding selected data structures and algorithms.
- Testing and troubleshooting application features.

Problem-Solving Skills

- Dividing a project into manageable modules.
- Designing recipe search and ingredient-matching workflows.
- Managing user interactions and persistent data.
- Identifying and resolving implementation errors.
- Understanding the importance of efficient algorithms.

Professional Skills

- Technical documentation and report preparation.
- Organizing project features and implementation details.
- Presenting and explaining a software project.
- Identifying limitations and planning future enhancements.

Through this project, practical knowledge of front-end development and algorithmic problem-solving was strengthened, while experience was gained in designing, implementing, testing, and documenting a software application.

REFERENCES

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

[2] Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

[3] W3Schools. *HTML Tutorial*. https://www.w3schools.com/html/

[4] W3Schools. *CSS Tutorial*. https://www.w3schools.com/css/

[5] MDN Web Docs. *JavaScript Guide*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[6] Oracle. *Java Documentation*. https://docs.oracle.com/en/java/

[7] Ricci, F., Rokach, L., and Shapira, B. (Eds.). (2022). *Recommender Systems Handbook* (3rd ed.). Springer.

[8] Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction, 12*, 331–370. https://doi.org/10.1023/A:1021240730564

[9] GeeksforGeeks. *Data Structures and Algorithms*. https://www.geeksforgeeks.org/dsa/

[10] MDN Web Docs. *Window: localStorage property*. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[11] Oracle. *Java Collections Framework*. https://docs.oracle.com/javase/8/docs/technotes/guides/collections/

[12] Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.